UNIVERSIDADE ESTADUAL DA PARAÍBA

CENTRO DE CIÊNCIAS E TECNOLOGIA - CCT

DEPARTAMENTO DE COMPUTAÇÃO - DC

JOHNATAN RODRIGUES DOS SANTOS

JOHNATTAN JOSE DA SILVA SANTOS

LÍVIA NASCIMENTO OLIVEIRA

PROBLEMA DA PARTIÇÃO DE CONJUNTOS

CAMPINA GRANDE - PB

2026

SUMÁRIO

1. INTRODUÇÃO..... 3

1.1 Contextualização do problema.....4

2. METODOLOGIA E IMPLEMENTAÇÃO..... 5

2.1 Arquitetura da Solução e Ferramentas..... 5

2.2 Pré-processamento e Estruturas de Dados..... 5

2.3 Modelagem de Passagem e Estado das Instâncias..... 6

3. ALGORITMOS IMPLEMENTADOS.....7

3.1 Backtracking.....7

3.1.1. Análise de Corretude..... 7

3.1.2. Análise de Desempenho Computacional.....8

3.1.2.1. Complexidade Espacial.....8

3.1.2.2. Temporal.....8

3.2. Branch and Bound..... 8

3.2.1. Análise de Corretude..... 8

3.2.2. Análise de Desempenho Computacional.....9

3.2.2.1. Complexidade Espacial.....9

3.2.2.2. Complexidade Temporal.....10

4 EXPERIMENTOS E RESULTADOS..... 10

4.1 Instâncias de Teste..... 10

4.2 Resultados Obtidos..... 11

5. ANÁLISE E DISCUSSÃO..... 11

5.1 Comparação das Estratégias: Backtracking Clássico vs. Branch and Bound 11

6. CONCLUSÃO..... 12

7. REFERÊNCIAS.....13

1. INTRODUÇÃO

1.1 Contextualização do problema

O Problema da Partição de Conjuntos (*Set Partition Problem*) é um dos problemas clássicos da teoria da complexidade computacional e da otimização combinatória. Ele consiste em um problema de decisão definido da seguinte forma: dado um multiconjunto $S = \{a_1, a_2, \ldots, a_n\}$ composto por inteiros estritamente positivos, deseja-se determinar se é possível particioná-lo em dois subconjuntos disjuntos, S_1 e S_2, de modo que a soma dos elementos de ambos seja exatamente igual.

Formalmente, o problema pode ser descrito pelas seguintes condições:

$S1 \cup S2 = S$, isto é, a união dos subconjuntos corresponde ao conjunto original;

$S1 \cap S2 = \emptyset$, ou seja, os subconjuntos são mutuamente exclusivos;

$\sum_{x \in S1} x = \sum_{y \in S2} y$, garantindo a igualdade entre as somas dos elementos.

Esse problema pode ser visto como um caso particular do Problema da Soma de Subconjuntos (*Subset Sum Problem*). Historicamente, ele ganhou destaque ao integrar o conjunto dos 21 problemas clássicos apresentados por Richard Karp em 1972, nos quais foi demonstrado que pertencem à classe dos problemas NP-completos (KARP, 1972). Pertencer à classe NP-completo implica que, embora seja possível verificar a validade de uma solução em tempo polinomial, não há, até o momento, conhecimento de um algoritmo determinístico que resolva o problema de forma geral também em tempo polinomial. Isso torna sua resolução computacionalmente desafiadora, especialmente para instâncias de grande porte. Diante dessa complexidade, abordagens ingênuas baseadas em força bruta, que avaliam todas as 2^n possíveis combinações, tornam-se rapidamente inviáveis. Assim, torna-se fundamental o uso de técnicas mais sofisticadas que reduzam o espaço de busca, como estratégias de poda e limitação. Nesse contexto, este

trabalho tem como objetivo explorar e analisar a aplicação de duas abordagens algorítmicas clássicas para a resolução do problema: o método *Backtracking* e a técnica de *Branch and Bound*.

2. METODOLOGIA E IMPLEMENTAÇÃO

2.1 Arquitetura da Solução e Ferramentas

O desenvolvimento algorítmico do presente projeto foi realizado na linguagem de programação Java, com o objetivo de alcançar elevado desempenho por meio do paradigma de orientação a objetos e da tipagem forte. A arquitetura da solução foi estruturada com base no princípio da separação de responsabilidades, promovendo maior coesão e organização entre os componentes do sistema.

A estrutura do sistema foi dividida nos seguintes módulos:

- **Manipulação de entradas e saídas (*FileHandler* e *Main*):** responsável pela leitura de dados a partir de arquivos no formato .txt, bem como pela formatação da saída contendo as partições resultantes, conforme o modelo estabelecido;
- **Modelagem de resposta (*PartitionResult*):** consiste em um objeto imutável que encapsula o resultado da execução, contendo um valor booleano indicativo de viabilidade e, quando aplicável, as duas partições resultantes;
- **Camada de resolução lógica (*PartitionSolver*):** representa o núcleo do sistema, no qual são implementadas as estratégias algorítmicas baseadas em árvores de decisão binárias. Essa classe contempla as duas abordagens analisadas neste trabalho: o método de *Backtracking* e de *Branch and Bound.*

2.2 Pré-processamento e Estruturas de Dados

Para ambas as abordagens adotadas, a estratégia de força bruta é substituída por mecanismos de busca sistemática com aplicação de técnicas de poda (pruning). Com o intuito de aumentar a eficiência dessas podas, foi implementada uma etapa de pré-processamento dos dados inicialmente, considera-se K como a soma total dos elementos do multiconjunto S, conforme a expressão a seguir:

$$K = \sum_{i=1}^{n} a_i$$

A primeira verificação baseia-se no princípio da paridade. Como o problema exige a divisão do conjunto em dois subconjuntos com somas iguais, é necessário que o valor total K seja par. Caso K mod 2 ≠ 0, conclui-se imediatamente pela impossibilidade de solução, evitando a execução de etapas computacionalmente custosas. Quando a condição de paridade é satisfeita, define-se um valor-alvo (*targetSum*) igual a $\frac{K}{2}$. Essa estratégia reduz o problema a uma única condição de verificação: encontrar um subconjunto S_1 cuja soma seja exatamente igual ao valor-alvo. Uma vez atingido esse valor, o subconjunto complementar S_2 automaticamente satisfaz as condições do problema.

2.3 Modelagem de Passagem e Estado das Instâncias

No que se refere ao gerenciamento de memória, optou-se pela utilização de estruturas do tipo *ArrayList*, passadas por referência ao longo das chamadas recursivas. Essa abordagem resulta em um consumo espacial de ordem O(n), sendo eficiente para o contexto do problema. Os subconjuntos em construção, representados por *currentSubset1* e *currentSubset2*, comportam-se como uma pilha, na qual elementos são inseridos durante a exploração de um ramo da árvore de decisão e removidos no processo de retorno da recursão (*backtracking*), permitindo a exploração de novas possibilidades. Essa estratégia de manipulação por referência é fundamental para evitar a realização de cópias profundas (*deep copy*), que poderiam impactar negativamente o desempenho, especialmente considerando a complexidade exponencial $O(2^n)$ do problema. A cópia das estruturas de dados é realizada apenas no momento em que uma solução válida é encontrada, garantindo maior eficiência no uso de memória.

3. ALGORITMOS IMPLEMENTADOS

Nesta seção, descreve-se a implementação das duas estratégias utilizadas para resolver o Problema da Partição: Backtracking e Branch and Bound. Ambas residem na classe PartitionSolver e operam sobre um conjunto de inteiros positivos.

3.1 Backtracking

A estratégia de busca com *Backtracking,* baseia-se em uma travessia em profundidade (*depth-first search*) em um espaço de estados implícito, modelado como uma árvore de decisão binária. Para cada elemento $a_i \in S_1$ são consideradas duas possibilidades:

- **Ramo de inserção:** o elemento a_i é incluído no subconjunto em construção S_1. Nesse caso, seu valor é somado à variável de controle *currentSum*, e a recursão avança para o próximo índice;
- **Ramo de exclusão:** o elemento a_i não é incluído em S_1, sendo implicitamente considerado como pertencente ao subconjunto complementar S_2. A recursão prossegue para o próximo índice sem alteração em *currentSum*.

3.1.1. Análise de Corretude

A corretude do algoritmo pode ser demonstrada a partir das seguintes propriedades:

- **Invariante de laço:** em qualquer nível recursivo iii, a variável currentSum representa corretamente a soma dos elementos presentes em S_1. Além disso, a união dos subconjuntos parciais S_1 e S_2, juntamente com os elementos ainda não processados $\{a_1, \ldots, a_n\}$, corresponde ao conjunto original;
- **Base da indução (i = 0):** no estado inicial, tem-se currentSum = 0, com subconjuntos vazios, satisfazendo a invariante;
- **Passo indutivo:** assumindo válida a invariante em um nível i, ao incluir a_i, atualiza-se corretamente currentSum. Caso contrário, ao não incluí-lo, o valor permanece inalterado. Em ambos os casos, a consistência do estado é preservada;
- **Conclusão:** ao atingir a condição currentSum = targetSum, garante-se que o subconjunto S_1 satisfaz o critério do problema. Considerando o espaço finito de 2^n possibilidades, conclui-se que o algoritmo é correto.

3.1.2. Análise de Desempenho Computacional

3.1.2.1. Complexidade Espacial

A complexidade de espaço do algoritmo é da ordem de O(n), uma vez que a profundidade máxima da recursão é proporcional ao número de elementos do conjunto. As estruturas utilizadas, como listas dinâmicas (*ArrayList*), contribuem para manter o uso de memória controlado.

3.1.2.2. Temporal

A complexidade temporal, no pior caso, é da ordem de $O(2^n)$. Isso ocorre porque o algoritmo pode explorar todas as combinações possíveis de inclusão e exclusão dos elementos. Na ausência de podas eficientes ou soluções encontradas precocemente, o crescimento do tempo de execução é exponencial em relação ao tamanho da entrada.

3.2. Branch and Bound

O algoritmo de Branch and Bound pode ser entendido como uma extensão otimizada da abordagem de Backtracking. Sua principal inovação consiste na introdução de funções de delimitação (bounds), responsáveis por estimar previamente a viabilidade de determinados ramos da árvore de decisão, permitindo a interrupção antecipada de caminhos que não levarão a uma solução válida.

3.2.1. Análise de Corretude

A corretude do algoritmo fundamenta-se no princípio de que apenas ramos comprovadamente inviáveis são descartados, garantindo que nenhuma solução válida seja perdida durante o processo de busca.

- **Invariante funcional:** um ramo da árvore de decisão é podado somente quando se comprova que não há possibilidade de atingir o valor-alvo targetSum a partir do estado atual. Dessa forma, a eliminação de caminhos inviáveis não compromete a integridade da busca por soluções corretas.

A estratégia de poda baseia-se na propriedade de que todos os elementos do conjunto são inteiros positivos, o que implica crescimento monotônico das somas parciais. A partir disso, definem-se dois critérios principais:

- **Poda por excesso (*overshoot pruning*):**

$$currentSum > targetSum$$

Quando a soma parcial excede o valor-alvo, conclui-se que não há possibilidade de ajuste futuro que reduza esse valor, uma vez que todos os elementos restantes são positivos. Assim, o ramo é imediatamente descartado.

- **Poda por insuficiência (*undershoot pruning*):**

$$currentSum + suffixSums[index] < targetSum$$

Nesse caso, mesmo considerando a soma de todos os elementos ainda não explorados (armazenados em suffixSums), verifica-se que não é possível atingir o valor-alvo. Dessa forma, o ramo também é descartado por inviabilidade.

Essas estratégias garantem que a busca seja significativamente reduzida, preservando apenas os caminhos potencialmente válidos.

3.2.2. Análise de Desempenho Computacional

3.2.2.1. Complexidade Espacial

A complexidade espacial do algoritmo permanece na ordem de O(n), semelhante à abordagem de Backtracking. Além das estruturas recursivas, utiliza-se um vetor auxiliar suffixSums, de tamanho proporcional a nnn, responsável por armazenar somas acumuladas dos elementos restantes. Ainda assim, o consumo total de memória mantém-se linear.

3.2.2.2. Complexidade Temporal

No pior caso, a complexidade temporal do algoritmo permanece exponencial, sendo limitada por $O(2^n)$. Esse cenário ocorre em instâncias construídas de forma a minimizar a eficácia das podas. Entretanto, na prática, observa-se uma redução significativa no tempo de execução médio (*average case*), uma vez que as

estratégias de poda eliminam grande parte das combinações inviáveis ainda nas fases iniciais da busca. Como resultado, o algoritmo apresenta desempenho substancialmente superior ao Backtracking puro em casos gerais.

4 EXPERIMENTOS E RESULTADOS

Com o objetivo de validar empiricamente as propriedades teóricas apresentadas na Seção 3, foi conduzido um conjunto de experimentos controlados. A análise concentrou-se na avaliação do crescimento assintótico e do tempo de execução (*runtime*) das abordagens de Backtracking e Branch and Bound, com ênfase no impacto das técnicas de poda na redução do espaço de busca.

4.1 Instâncias de Teste

Para simular diferentes níveis de complexidade computacional associados ao limite exponencial $O(2^n)$, foram definidas instâncias de teste com tamanhos progressivos. Essas instâncias foram organizadas em três categorias principais:

- **Instâncias triviais ($n \leq 20$)**: conjuntos pequenos de inteiros, utilizados para validação da corretude dos algoritmos e verificação da consistência das partições S_1 e S_2;

- **Instâncias de escalabilidade ($20 < n \leq 40$)**: conjuntos de tamanho intermediário, empregados para observar o crescimento do custo computacional e os efeitos da explosão combinatorial;

- **Casos extremos (*edge cases*)**: instâncias projetadas para acionar condições específicas do algoritmo, como:

    - somas totais ímpares ($K \bmod 2 \neq 0$), resultando em encerramento imediato no pré-processamento;

    - distribuições de valores que dificultam ou favorecem a atuação das heurísticas de poda.

4.2 Resultados Obtidos

Os resultados experimentais corroboram as expectativas teóricas estabelecidas na análise assintótica.

Nas instâncias triviais ($n \leq 20n$), ambas as abordagens apresentaram desempenho semelhante, com tempos de execução na ordem de milissegundos. Nesses casos, o número de combinações possíveis (2^n) ainda é suficientemente

pequeno para ser explorado rapidamente por hardware moderno, tornando pouco perceptível o impacto das otimizações.

Entretanto, à medida que o tamanho das instâncias aumenta, diferenças significativas de desempenho tornam-se evidentes.

- **Limitações do Backtracking:**
  A abordagem baseada exclusivamente em Backtracking apresentou crescimento exponencial acentuado a partir de $n > 35$. Na ausência de mecanismos de poda, o algoritmo percorre um grande número de combinações inviáveis, incluindo subconjuntos que, mesmo utilizando todos os elementos restantes, não seriam capazes de atingir o valor-alvo. Esse comportamento resulta em aumento expressivo no tempo de execução, podendo levar a situações de inviabilidade prática, como tempos excessivos de processamento ou interrupções por limite de tempo (*timeout*).

- **Eficiência do Branch and Bound:**
  Em contraste, o algoritmo de Branch and Bound demonstrou desempenho significativamente superior. A utilização de técnicas de poda, como verificação de excesso e insuficiência por meio de somas sufixas, permitiu eliminar antecipadamente grande parte dos ramos inviáveis da árvore de decisão. Como resultado, foi possível resolver instâncias de maior dimensão ($n \approx 50$, dependendo dos valores) em tempos reduzidos.

5. ANÁLISE E DISCUSSÃO

Os resultados empíricos apresentados na seção anterior corroboram as hipóteses teóricas estabelecidas durante a modelagem das abordagens. Embora os algoritmos de Backtracking e Branch and Bound compartilhem a mesma estrutura recursiva e apresentem consumo de memória linear $O(n)$, garantido pelo uso de estruturas por referência, observa-se uma diferença significativa no comportamento prático de execução, especialmente em relação à escalabilidade.

5.1 Comparação das Estratégias: Backtracking Clássico vs. Branch and Bound

Ao comparar as duas estratégias, verifica-se que ambas apresentam a mesma complexidade no pior caso, $O(2^n)$, característica inerente a problemas classificados como NP-completos. No entanto, a principal diferença entre os métodos reside na eficiência da exploração do espaço de busca, proporcionada pelo uso de heurísticas.

- **Eficiência versus ausência de heurística:**
  O algoritmo de Backtracking clássico realiza uma busca exaustiva sem

qualquer mecanismo de previsão sobre a viabilidade dos caminhos explorados. Dessa forma, percorre diversas combinações que poderiam ser descartadas antecipadamente. Em instâncias de pequeno porte, essa limitação não impacta significativamente o desempenho. Entretanto, à medida que o tamanho da entrada aumenta ($n \geq 30$), essa abordagem torna-se computacionalmente ineficiente, resultando em crescimento exponencial acentuado no tempo de execução.

- **Custo de pré-processamento (*trade-off*):**
  O método de Branch and Bound introduz um custo adicional de pré-processamento, correspondente ao cálculo das somas sufixas, com complexidade $O(n)$. Apesar desse custo inicial, os ganhos obtidos durante a execução compensam amplamente esse investimento. A utilização dessas informações permite a eliminação antecipada de ramos inviáveis, reduzindo significativamente o número de chamadas recursivas e, consequentemente, o tempo total de processamento.

- **Comportamento em diferentes distribuições de dados:**
  Em cenários nos quais elementos de maior valor aparecem nas primeiras posições, ambos os algoritmos tendem a se beneficiar da poda por excesso (currentSum > targetSum). No entanto, em distribuições mais equilibradas ou aleatórias, a poda por insuficiência (currentSum + suffixSums[index] < targetSum) torna-se determinante, favorecendo o desempenho do Branch and Bound.

Conclui-se que o algoritmo de Backtracking, embora conceitualmente relevante e adequado para fins didáticos ou para instâncias de pequeno porte, apresenta limitações significativas em termos de eficiência prática. Por outro lado, o método de Branch and Bound, ao incorporar heurísticas de poda, mantém a mesma complexidade teórica no pior caso, mas apresenta desempenho substancialmente superior em cenários reais.

6. CONCLUSÃO

O presente trabalho abordou o Problema da Partição de Conjuntos sob a perspectiva da otimização combinatória, com o objetivo de implementar, analisar e comparar duas técnicas clássicas de busca: o Backtracking e o método de Branch and Bound (Ramificar e Limitar). Ambas as abordagens foram implementadas na linguagem Java, utilizando estruturas recursivas e gerenciamento eficiente de memória com complexidade espacial $O(n)$. A análise teórica, aliada aos experimentos realizados, confirmou a corretude das abordagens e os limites

assintóticos esperados para problemas pertencentes à classe NP-completo. Em ambos os casos, a complexidade no pior cenário permanece exponencial, da ordem de $O(2^n)$, evidenciando a natureza computacionalmente desafiadora do problema.

Entretanto, os resultados práticos demonstraram diferenças significativas de desempenho entre as abordagens. O Backtracking, por não utilizar mecanismos de poda, apresentou crescimento exponencial acentuado, tornando-se inviável para instâncias de maior dimensão ($n \geq 35$). Por outro lado, o algoritmo de Branch and Bound, ao incorporar estratégias heurísticas de poda, como as verificações de excesso (currentSum > targetSum) e insuficiência (currentSum + suffixSums[index] < targetSum), reduziu consideravelmente o espaço de busca explorado.

Dessa forma, observou-se que o custo adicional de pré-processamento, necessário para o cálculo das somas sufixas, é amplamente compensado pela redução no número de chamadas recursivas. Essa otimização permitiu a resolução eficiente de instâncias significativamente maiores, evidenciando a eficácia do método em cenários práticos. Conclui-se, portanto, que abordagens puramente exaustivas são inadequadas para a resolução eficiente de problemas combinatórios de grande escala. A utilização de técnicas de poda, como as empregadas no Branch and Bound, mostra-se essencial para tornar viável a obtenção de soluções em tempo computacional aceitável, consolidando essa abordagem como a alternativa mais adequada para o problema estudado.

7. REFERÊNCIAS

KARP, Richard M. Reducibility among combinatorial problems. In: MILLER, Raymond E.; THATCHER, James W. (ed.). Complexity of Computer Computations. New York: Plenum Press, 1972.